Digital VLSI Design

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Dhriti Sharma , 2026203015
Achanti Pavan Kumar, 2026203016

Project link : https://github.com/Pavan-Achanti/DVD_assgn

Questions for the Report

1)The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply 16+12.

A) Synthesis produced 63 cells in total, including 16 flip-flops, so the combinational logic consists of 63 − 16 = **47 cells**. This is not simply 16 + 12 = 28 because the 16 AND gates and 12 full adders describe the logical components of the multiplier, whereas synthesis maps their Boolean functions into available SKY130 standard cells. A full adder may require multiple standard cells, and synthesis can optimize, combine, or restructure logic; therefore, the mapped cell count differs from the number of logical components.

2)What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about 2N full-adder delays?

A) The critical path runs from register `_095_` to register `_092_` and contains **10 cells**, including the launch and destination flip-flops. This is reasonably consistent with the estimate of approximately 2N full-adder delays for an N-bit array multiplier: for N = 4, the estimate is about 2 × 4 = 8 full-adder delays. The actual path contains a mixture of standard-cell types rather than eight complete full adders, so the two counts are only approximate comparisons.

3)What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

A) The 16 flip-flops occupy 400.384 μm² out of the total synthesized cell area of 740.7104 μm². Therefore, the flip-flop area percentage is (400.384 / 740.7104) × 100 ≈ **54.05%**. This percentage is large because the design registers both 4-bit inputs and the 8-bit output, requiring 16 flip-flops, and the area of these storage cells is substantial relative to the small amount of combinational logic in this design.

4) Why is the post-route Fmax lower than the Fmax from Part C? Which kinds of cells did the flow add, and why is each kind needed?

A) The estimated post-route maximum frequency is approximately **256.15 MHz**, lower than the Part C estimate of 384.02 MHz, because physical implementation introduces effects such as routing parasitics and additional delay from physical cells. The flow increased the standard-cell count from 63 after synthesis to 121 after place and route. It added timing-repair buffers to improve timing, clock buffers to distribute the clock signal, and hold buffers to correct hold-time violations. The flow also inserted fill cells to satisfy layout and manufacturing requirements; these fill cells are not functional logic.

5) What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

A) Design Rule Checking (DRC) verifies that the physical layout follows the foundry's geometric and manufacturing rules, such as minimum spacing and width requirements. Layout Versus Schematic (LVS) checks whether the electrical connectivity extracted from the layout matches the intended circuit netlist. Both checks are necessary because a layout may satisfy geometric rules while having incorrect electrical connections, or it may have correct connectivity while violating manufacturing rules. In the final run, the reported DRC count, and LVS count were all **zero**.

6) Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

A) According to Section 2, an N-bit array multiplier uses N^2 AND gates and $N(N-1)$ full adders, and its longest path is approximately 2N full-adder delays. For a 4-bit multiplier, the estimated number of full adders is $4(4-1) = 12$.. For an 8-bit multiplier, the estimated number is $8(8-1) =$ **56 full adders**, while the number of AND gates increases from $4^2 = 16$ to $8^2 =$ **64**. The estimated critical path increases from approximately $2 \times 4 = 8$ full-adder delays to $2 \times 8 =$ **16 full-adder delays**. Therefore, doubling the multiplier width increases the estimated full-adder count by $56/12 \approx$ **4.67 times**, while the critical-path delay approximately doubles, assuming similar full-adder delays.

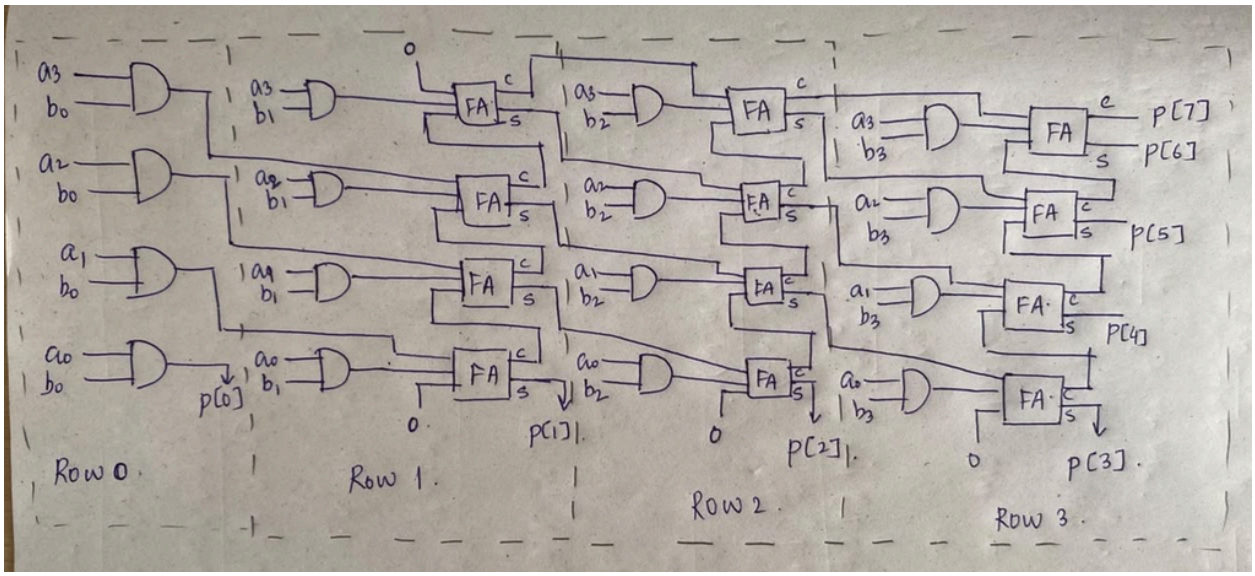
Detailed report

Part A: RTL Design and Verification

Design Overview

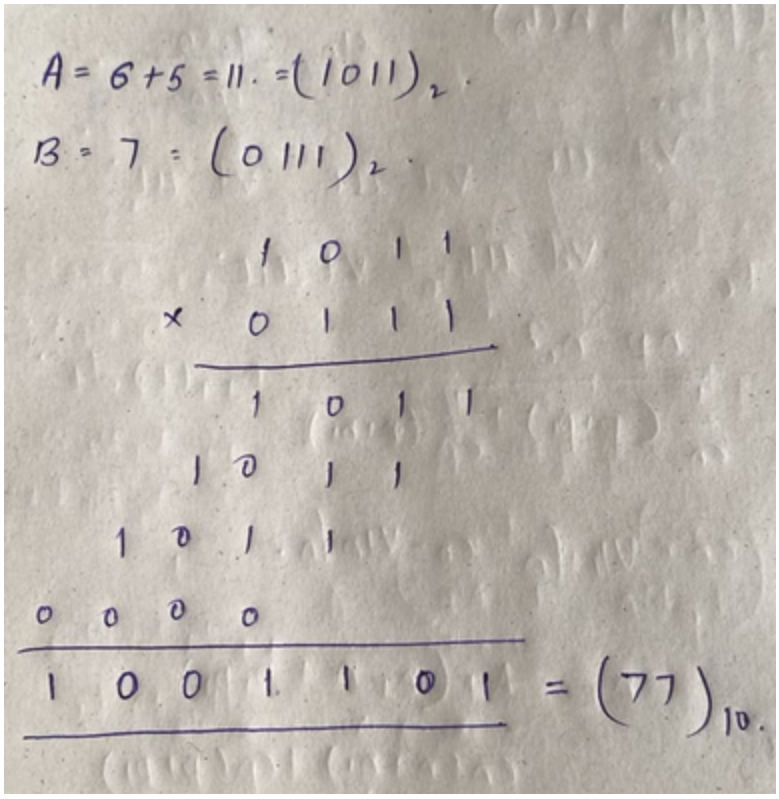
We created a 4-bit unsigned array multiplier that computes an 8-bit output.

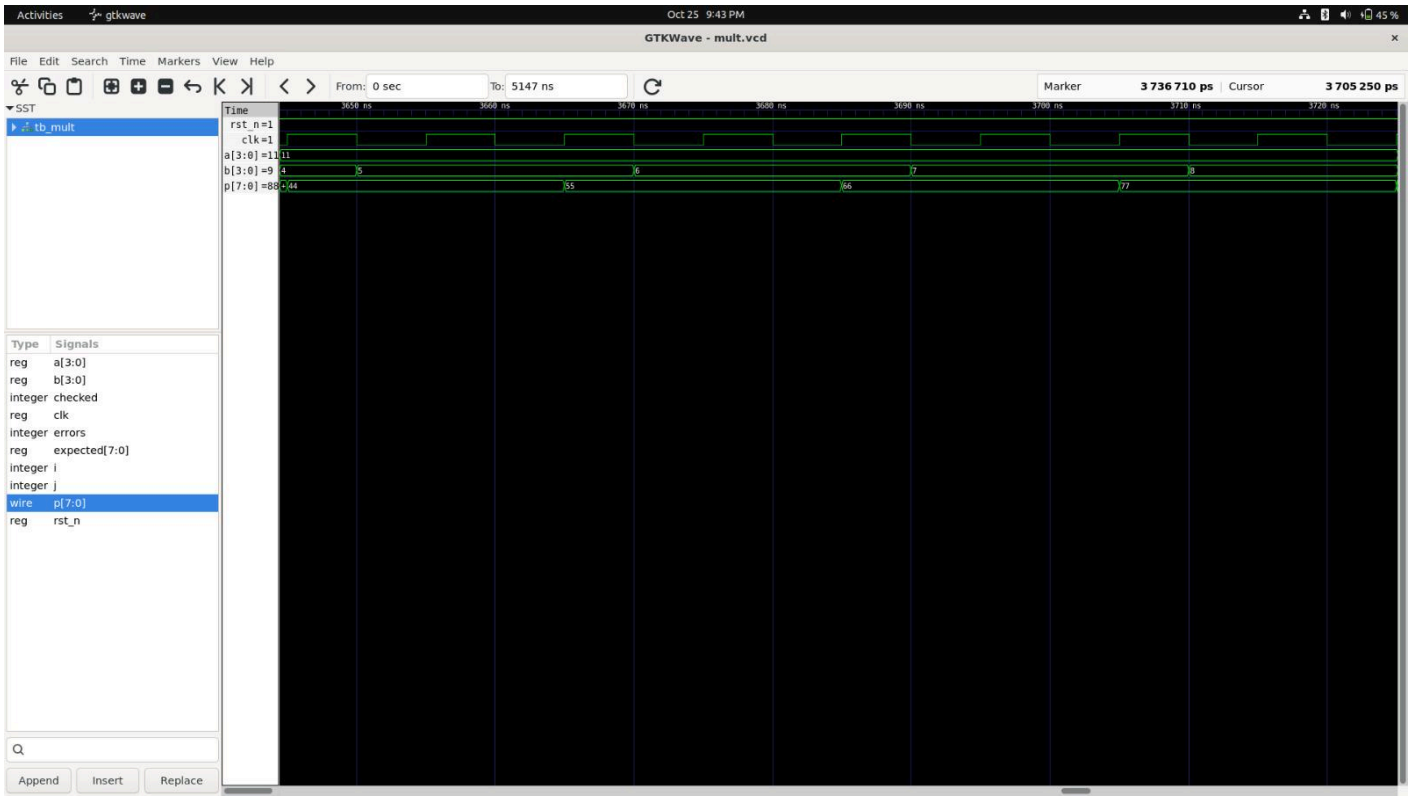
- This design needs 16 AND gates physically to generate the partial products (the multiplication-in-between) and 12 full adders to add the partial products together over 3 rows.
- The design has a total of 16 flip-flops (registers) in use i.e. 4 for input a, 4 for input b and 8 to store final output p.
- As inputs wait for one clock edge to get stored and output is latched on the next clock edge, the proposed design has a latency of exactly 2 clock cycles.



Hand-Worked Example

To verify the multiplier, we selected custom inputs as given in the specifications.


A = 6+5 = 11. = (1011)2.
B = 7 = (0111)2.
1 0 1 1
× 0 1 1 1
1 0 1 1
1 0 1 1
1 0 1 1
0 0 0 0
1 0 0 1 1 0 1 1 = (77)10.


Activities
gtkwave
Oct 25 9:43 PM
45 %
GTKWave - mult.vcd
File Edit Search Time Markers View Help
From: 0 sec
To: 5147 ns
Marker 3 736 710 ps
Cursor 3 705 250 ps
SST
tb_mult
Type Signals
reg a[3:0]
reg b[3:0]
integer checked
reg clk
integer errors
reg expected[7:0]
integer i
integer j
wire p[7:0]
reg rst_n
Time
rst_n=1
clk=1
a[3:0]=11
b[3:0]=9
p[7:0]=88
3650 ns
3660 ns
3670 ns
3680 ns
3690 ns
3700 ns
3710 ns
3720 ns
11
4
5
6
7
8
44
55
66
77
Append
Insert
Replace

Simulation and Verification

We added a testbench (tb_mult.v) to verify all 256 A and B combinations. After compilation of the testbench mult, it issued no errors in return, thus confirming that our verilog code is correct.

- Command used for Compliation: iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
- Testbench Pass Output:
  VCD info: dumpfile mult.vcd opened for output.
  PASS: 256 vectors checked, 0 errors
  tb/tb_mult.v:97: $finish called at 5146000 (1ps)
- Fault Injection Test: To make sure the testbench will catch mistakes (if any), we replaced a^b^cin with a^b in full_adder.v. The following output was given after running the testbench.
- Testbench Fail Output:

VCD info: dumpfile mult.vcd opened for output.
MISMATCH: a=3 b=3 expected=9 got=5
MISMATCH: a=3 b=6 expected=18 got=10
MISMATCH: a=3 b=7 expected=21 got=9
MISMATCH: a=3 b=11 expected=33 got=29
MISMATCH: a=3 b=12 expected=36 got=20
MISMATCH: a=3 b=13 expected=39 got=23
MISMATCH: a=3 b=14 expected=42 got=18
MISMATCH: a=3 b=15 expected=45 got=17
MISMATCH: a=5 b=5 expected=25 got=17
MISMATCH: a=5 b=7 expected=35 got=27
MISMATCH: a=5 b=10 expected=50 got=34
MISMATCH: a=5 b=11 expected=55 got=39
MISMATCH: a=5 b=13 expected=65 got=57
MISMATCH: a=5 b=14 expected=70 got=54
MISMATCH: a=5 b=15 expected=75 got=51
MISMATCH: a=6 b=3 expected=18 got=10
MISMATCH: a=6 b=6 expected=36 got=20
MISMATCH: a=6 b=7 expected=42 got=18
MISMATCH: a=6 b=11 expected=66 got=58
MISMATCH: a=6 b=12 expected=72 got=40
MISMATCH: a=6 b=13 expected=78 got=46
MISMATCH: a=6 b=14 expected=84 got=36
MISMATCH: a=6 b=15 expected=90 got=34
MISMATCH: a=7 b=3 expected=21 got=9
MISMATCH: a=7 b=5 expected=35 got=27
MISMATCH: a=7 b=6 expected=42 got=18
MISMATCH: a=7 b=7 expected=49 got=21
MISMATCH: a=7 b=10 expected=70 got=54
MISMATCH: a=7 b=11 expected=77 got=49
MISMATCH: a=7 b=12 expected=84 got=36

MISMATCH: a=7 b=13 expected=91 got=35
MISMATCH: a=7 b=14 expected=98 got=42
MISMATCH: a=7 b=15 expected=105 got=45
MISMATCH: a=9 b=9 expected=81 got=65
MISMATCH: a=9 b=11 expected=99 got=83
MISMATCH: a=9 b=13 expected=117 got=101
MISMATCH: a=9 b=15 expected=135 got=247
MISMATCH: a=10 b=5 expected=50 got=34
MISMATCH: a=10 b=7 expected=70 got=118
MISMATCH: a=10 b=10 expected=100 got=68
MISMATCH: a=10 b=11 expected=110 got=78
MISMATCH: a=10 b=13 expected=130 got=114
MISMATCH: a=10 b=14 expected=140 got=236
MISMATCH: a=10 b=15 expected=150 got=166
MISMATCH: a=11 b=3 expected=33 got=61
MISMATCH: a=11 b=5 expected=55 got=39
MISMATCH: a=11 b=6 expected=66 got=122
MISMATCH: a=11 b=7 expected=77 got=81
MISMATCH: a=11 b=9 expected=99 got=83
MISMATCH: a=11 b=10 expected=110 got=78
MISMATCH: a=11 b=11 expected=121 got=229
MISMATCH: a=11 b=12 expected=132 got=244
MISMATCH: a=11 b=13 expected=143 got=127
MISMATCH: a=11 b=14 expected=154 got=162
MISMATCH: a=11 b=15 expected=165 got=137
MISMATCH: a=12 b=3 expected=36 got=52
MISMATCH: a=12 b=6 expected=72 got=104
MISMATCH: a=12 b=7 expected=84 got=68
MISMATCH: a=12 b=11 expected=132 got=212
MISMATCH: a=12 b=12 expected=144 got=208
MISMATCH: a=12 b=13 expected=156 got=220
MISMATCH: a=12 b=14 expected=168 got=136
MISMATCH: a=12 b=15 expected=180 got=164
MISMATCH: a=13 b=3 expected=39 got=55
MISMATCH: a=13 b=5 expected=65 got=121
MISMATCH: a=13 b=6 expected=78 got=110
MISMATCH: a=13 b=7 expected=91 got=67
MISMATCH: a=13 b=9 expected=117 got=101
MISMATCH: a=13 b=10 expected=130 got=242
MISMATCH: a=13 b=11 expected=143 got=223
MISMATCH: a=13 b=12 expected=156 got=220
MISMATCH: a=13 b=13 expected=169 got=145
MISMATCH: a=13 b=14 expected=182 got=134
MISMATCH: a=13 b=15 expected=195 got=171

```
MISMATCH: a=14 b=3 expected=42 got=50
MISMATCH: a=14 b=5 expected=70 got=118
MISMATCH: a=14 b=6 expected=84 got=100
MISMATCH: a=14 b=7 expected=98 got=74
MISMATCH: a=14 b=10 expected=140 got=236
MISMATCH: a=14 b=11 expected=154 got=194
MISMATCH: a=14 b=12 expected=168 got=200
MISMATCH: a=14 b=13 expected=182 got=134
MISMATCH: a=14 b=14 expected=196 got=148
MISMATCH: a=14 b=15 expected=210 got=186
MISMATCH: a=15 b=3 expected=45 got=49
MISMATCH: a=15 b=5 expected=75 got=115
MISMATCH: a=15 b=6 expected=90 got=98
MISMATCH: a=15 b=7 expected=105 got=77
MISMATCH: a=15 b=9 expected=135 got=247
MISMATCH: a=15 b=10 expected=150 got=230
MISMATCH: a=15 b=11 expected=165 got=201
MISMATCH: a=15 b=12 expected=180 got=196
MISMATCH: a=15 b=13 expected=195 got=139
MISMATCH: a=15 b=14 expected=210 got=154
MISMATCH: a=15 b=15 expected=225 got=181
FAIL: 256 vectors checked, 95 errors
tb/tb_mult.v:97: $finish called at 5146000 (1ps)
```

Part B: Logic Synthesis with Yosys

Synthesis Procedure

The synthesis flow was automated using a Tcl script, `synth/synth.tcl`. The RTL files `full_adder.v` and `mult_array.v` were read into Yosys, and the multiplier was synthesized with `mult_array` as the top-level module. The design was flattened, and the generic flip-flops and combinational logic were mapped to cells from the SKY130 high-density standard-cell library using `dfflibmap` and ABC technology mapping. The resulting gate-level netlist was saved as `mult_array_netlist.v`, and the synthesis statistics were written to `mult_array_stat.rpt`.

The synthesized netlist was subsequently simulated using the same testbench used in Part A. The testbench checked all 256 combinations of the two 4-bit inputs.

Synthesis Results

The synthesis results are summarized below.

Metric	Result
Total number of cells	63
Number of flip-flops	16
Number of non-flip-flop cells	47
Total cell area	740.7104 μm²
Flip-flop area	400.384 μm²
Flip-flop area as a percentage of total area	54.05%
Gate-level simulation	PASS

The most frequently occurring standard cells in the synthesized netlist were:

Standard-cell type	Count
`sky130_fd_sc_hd__dfrtp_1`	16
`sky130_fd_sc_hd__nand2_1`	8
`sky130_fd_sc_hd__a21oi_1`	6

The 16 flip-flops correspond to the four registered bits of each input and the eight registered output bits. The remaining 47 cells implement the combinational logic required for multiplication. Although the multiplier architecture uses 16 AND gates and 12 full adders, synthesis does not necessarily preserve these as separate physical cells. Instead, the logic is optimized and mapped into available standard cells, so the resulting cell count differs from the number of components in the RTL architecture.

Flip-flops account for approximately 54.05% of the total cell area. This relatively large fraction is expected for a small multiplier because the design contains 16 registers, while the combinational logic is comparatively small. Each register requires a physical sequential cell, which occupies more area than many simple logic gates.

Gate-Level Verification

The synthesized netlist was verified using gate-level simulation. The testbench exhaustively tested all 256 possible combinations of the two 4-bit inputs and compared the registered output against the expected product, respecting the design's two-clock-cycle latency.

The simulation completed successfully and reported:

```
PASS: 256 vectors checked, 0 errors
```

This result confirms that the synthesized netlist produces the expected multiplication results for all tested input combinations.

Conclusion

The 4-bit array multiplier was successfully synthesized into a SKY130 standard-cell netlist containing 63 cells, including 16 flip-flops. The total mapped cell area was 740.7104 μm², with flip-flops contributing 54.05% of the total area. Successful exhaustive gate-level simulation verified the functional correctness of the synthesized implementation.

Part C: Static Timing Analysis (OpenSTA)

Timing Results Table

Using OpenSTA and the SkyWater 130nm standard cell library, we checked how fast the chip can run with signals not arriving too late (setup slack).

Metric	**Value**
Worst Slack at T=10ns	7.396ns
Data Arrival TIme	2.471ns
Data Required Time	9.866ns
Tmin (Minimum Clock Period)	2.604ns (Rounded to 2.60ns)
Fmax (Maximum Frequency)	384.02MHz
Slack at T=2.60ns	-0.004ns

Critical path start point	Register *095*
Critical path end point	Register *092*
Number of cells on critical path	10 (2 flip-flops + 8 combinational logic gates)
Latency	2*2.60ns=5.20ns
Throughput	384.02 million results per second

Critical Path and Standard Cell Analysis

The critical path is the longest path that an electrical signal travels from the input flip-flop (_095_) to the output flip-flop (_092_). Yosys synthesis simplified our design, not chaining literal full adders, and compressed the math into 10 highly optimized SkyWater 130nm physical components to minimize delay. The standard cells met along this path are:

- sky130_fd_sc_hd__dfrtp_1 (Flip Flops): Asynchronous reset memory cell with active low reset. It appears twice, once at the input (_095_) and once at the output where we get the final computed result (_092_).
- sky130_fd_sc_hd__clkinv_1 (Inverter) : Basic NOT gate that inverts the input signal.
- sky130_fd_sc_hd__o311ai_0 sky130_fd_sc_hd__a21oi_1 (Logic Complex) These are OR-AND-Invert and AND-OR-Invert gates. Instead of sending signals through separate sequential gates, these “super-cells” perform multiple logic operations simultaneously, at the hardware level.
- sky130_fd_sc_hd__maj3_1 (Majority Gate) This is a very specialized cell, and it will produce a ' 1' if and only if more than half of the inputs (2 of the 3) are '1'. It calculates the “carry” bit in addition much quicker than ordinary gates.
- sky130_fd_sc_hd__xnor2_1 & sky130_fd_sc_hd__xnor3_1 (XNOR Gates) : These verify that signals are even or odd, or that they are equal to each other. They are the basic cells that compute the final “sum” bit when adding binary numbers.

Comparison with Architectural Estimate

The initial theoretical estimate gave a critical path delay of approximately 8 full-adder delays (2N, with N=4). The total measured logic delay, however, was only 2.471 ns because the design was reduced and mapped into optimized standard cells (such as majority gates and complex logic) instead of discrete full-adder blocks.

Part D — Physical Design: RTL to GDSII

Physical-Design Procedure

The physical-design flow was configured using `pnr/config.json`. The design name was set to `mult_array`, the RTL source files were specified, and `clk` was selected as the clock port. The die dimensions were fixed at 150 μm × 150 μm, corresponding to a die area of 22,500 μm².

Based on the minimum clock period obtained in Part C, the clock period was initially set to 3.5 ns. The initial run exhibited a small negative setup slack at the slow timing corner. The clock period was therefore increased to 4.0 ns.

The final 4.0 ns run completed with zero reported DRC errors, zero LVS errors, and non-negative setup slack at all three reported timing corners. The final metrics were saved in `pnr/metrics.json`, and the generated layout was saved as `pnr/mult_array.gds`. KLayout was used to inspect the complete layout and a zoomed-in view of the standard cells and routing.

Physical-Design Results

The results obtained from the final 4.0 ns run are summarized below.

Metric	Result
Clock period	4.0 ns
Number of runs needed	2(one for 3.5 ns and one for 4.0ns)
Die area	22,500 μm²
Core area	16,199 μm²
Cells after synthesis (Part B)	63
Standard cells after place and route	121
Final routed wirelength	2,979 μm
Worst setup slack	+0.096035 ns
Estimated post-route maximum frequency	256.15 MHz
Maximum frequency from Part C	384.02 MHz
DRC errors	0
LVS errors	0

The core occupies approximately 72.0% of the total die area. The reported standard-cell utilization is approximately 12.62%, indicating that the placed standard cells occupy a relatively small fraction of the available core area.

Timing Analysis

The worst setup slack reported at the slow timing corner was +0.096035 ns, equivalent to approximately 96 ps. The setup-violation count was zero at the fast, typical, and slow timing corners. Therefore, the final design meets the configured 4.0 ns setup timing constraint.

The post-route maximum frequency was estimated using the following formula:

$$F_{max,post} = 1000 / (T_{clock} - WNS)$$

where T_{clock} is the clock period in nanoseconds and WNS is the worst setup slack in nanoseconds.

Substituting the measured values:

$$T_{min,post} = 4.0 - 0.096035 = 3.903965 \text{ ns}$$

Therefore,

$$F_{max,post} = 1000 / 3.903965 \approx 256.15 \text{ MHz}$$

The estimated post-route maximum frequency is lower than the 384.02 MHz obtained in Part C. Physical implementation introduces clock-tree effects, interconnect parasitics, and additional timing constraints that are not fully represented in the earlier simplified static timing analysis. Consequently, the physical-design timing estimate provides a more implementation-aware assessment of the circuit's performance.

Additional Cells Introduced During Physical Design

The physical-design flow introduced additional cells to support timing closure, clock distribution, and layout fabrication requirements.

Cell category	Count	Purpose
Timing-repair buffers	33	Improve signal drive and help satisfy timing constraints
Clock buffers	5	Distribute the clock signal to sequential cells

Hold buffers	3	Correct minimum-delay violations
Fill cells	1,209	Support layout-density and manufacturing requirements

The number of standard cells increased from 63 after synthesis to 121 after placement and routing. This increase reflects physical optimization and the insertion of cells required for clock distribution and timing repair. Fill cells are reported separately from the standard-cell count and do not represent additional functional multiplier logic.

Physical Verification

The final layout was checked using design-rule checking (DRC) and layout-versus-schematic (LVS) verification.

DRC checks whether the physical layout satisfies the geometric and manufacturing constraints specified by the process design kit, including rules related to wire widths, spacing, and layer interactions. The final metrics reported zero DRC errors in both Magic and KLayout.

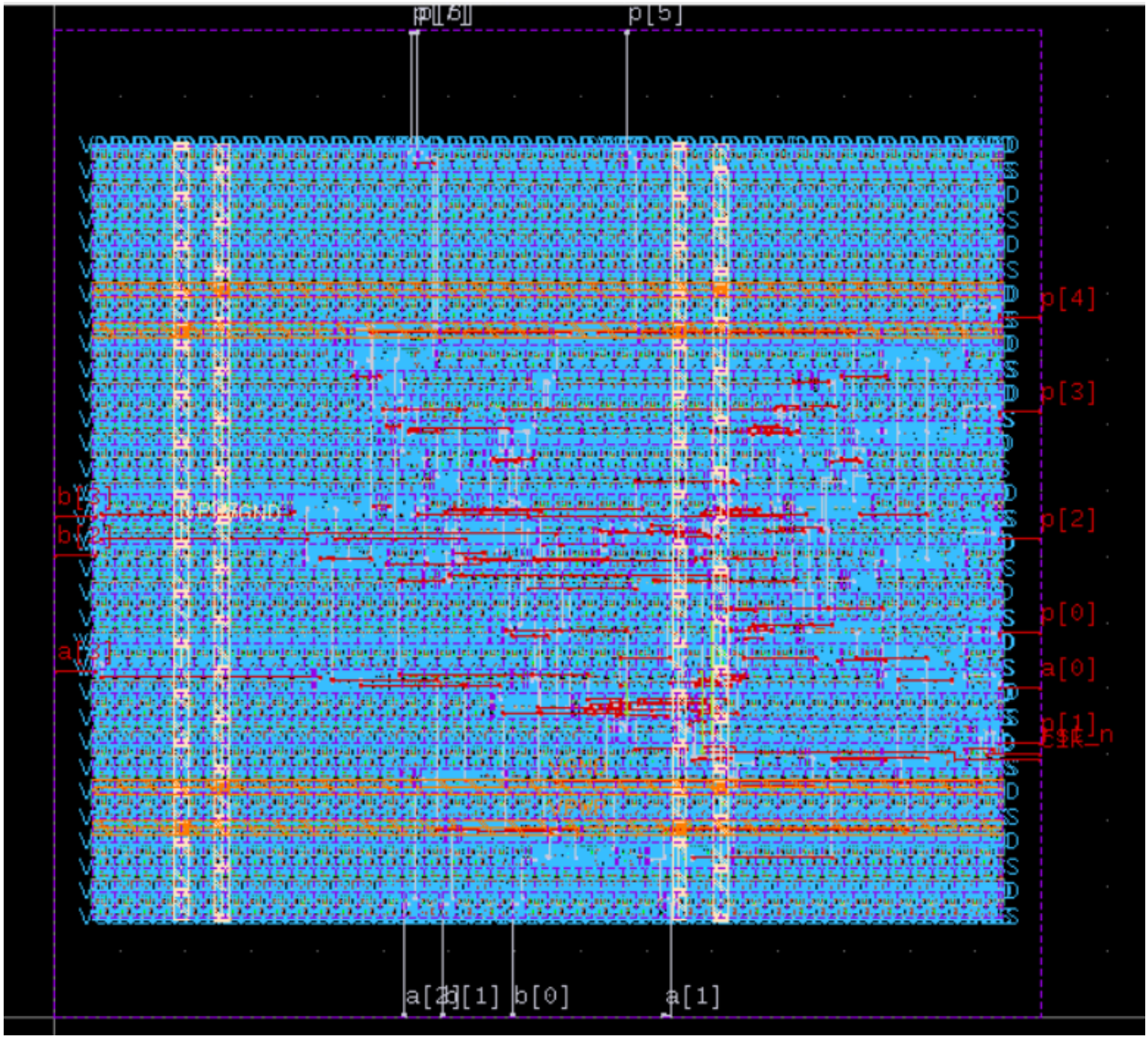
LVS compares the circuit extracted from the physical layout against the intended circuit netlist. The final metrics reported zero LVS errors, with no reported device, net, or pin mismatches.

Passing both checks provides evidence that the generated layout satisfies the reported physical-design rules and corresponds to the intended circuit. These checks are essential because a layout that appears visually correct may still contain manufacturing-rule violations or electrical connectivity errors.

Layout Inspection

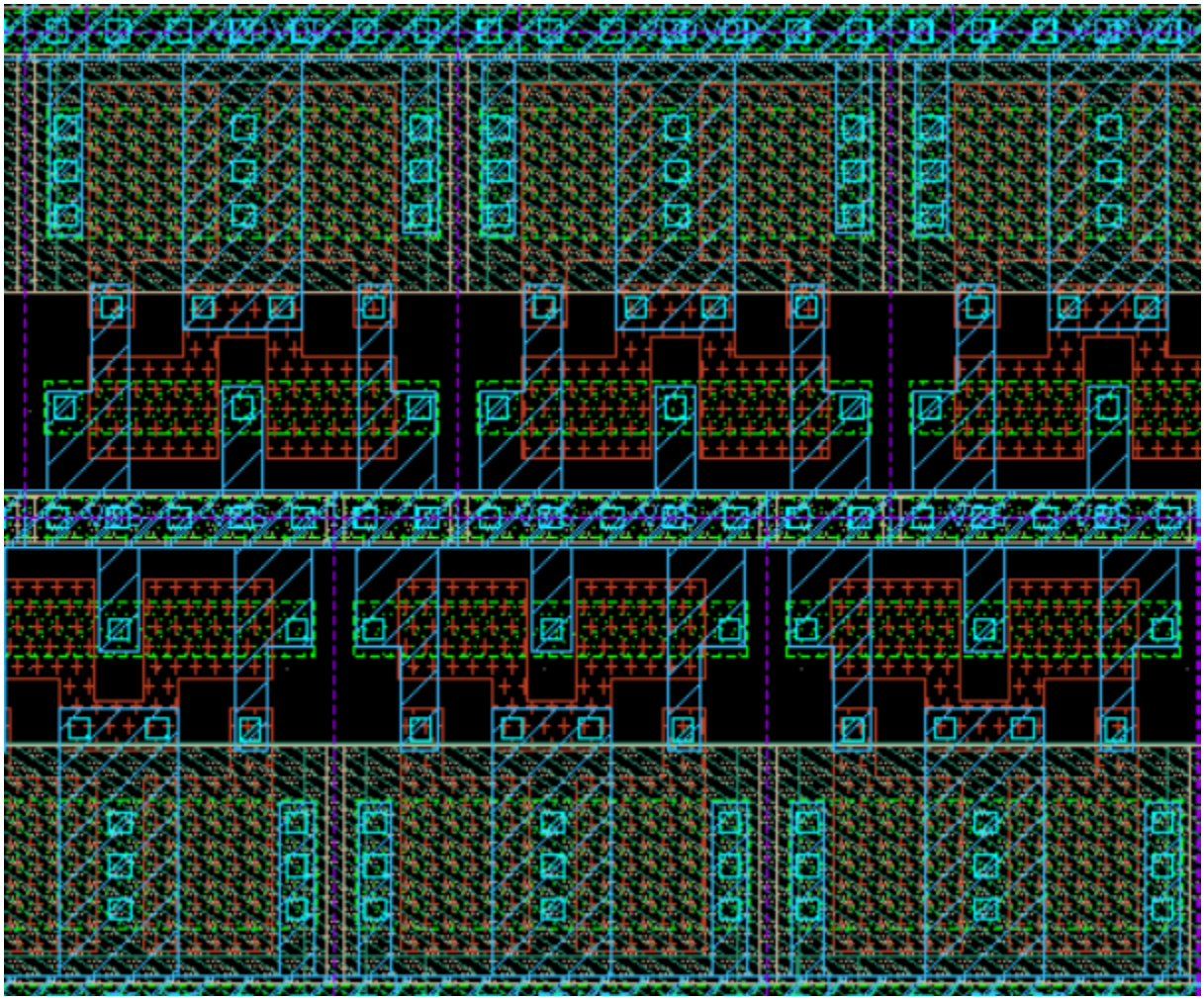
The final GDSII layout was opened in KLayout for visual inspection.

Figure D.1: Full-chip layout of the 4-bit array multiplier.



The full-layout view shows the physical extent of the design, its standard-cell placement, routing structures, and external pins.

Figure D.2: Zoomed-in view of standard cells and metal interconnects.



The zoomed-in view shows the repeated standard-cell structures and the metal interconnects used to connect the logic elements.

Conclusion

The 4-bit array multiplier was successfully taken through the physical-design flow using LibreLane and the SKY130 process design kit. The final implementation used a 4.0 ns clock period and achieved a worst setup slack of +0.096035 ns at the slow timing corner. The estimated post-route maximum frequency was 256.15 MHz. The final layout had a core area of 16,199 μm² and a routed wirelength of 2,979 μm. Both DRC and LVS reported zero errors, demonstrating that the layout passed the reported physical-verification checks. The generated GDSII file was inspected in KLayout to examine the complete chip layout and the standard-cell interconnections.

Individual Contributions

Dhriti Sharma: Part A, Part C

Achanti Pavan Kumar: Part B, PartD